Dual Motor Closed-Loop PID Speed Control for a Mobile Robot Platform

Name Abha Singh Sardar

SR Number 27086

Course MN 207: Mobile Robots

Module Optical Feedback and Discrete Dual PID Control

October 22, 2026

1 Introduction

1.1 Aim

The aim of this laboratory assignment is to design, implement, and validate dual independent closed-loop Proportional-Integral-Derivative (PID) speed controllers on an Arduino Mega 2560 microcontroller to regulate the rotational velocity of two direct current (DC) gear motors. Optical wheel encoders provide real-time pulse feedback, allowing the firmware to dynamically modulate Pulse Width Modulation (PWM) duty cycles so that both motors converge to and sustain an identical target speed of 200.0 RPM despite substantial electromechanical asymmetries.

1.2 Overview of Approach and Control Motivation

In differential-drive robotics, uncompensated actuator differences cause severe yaw and directional drift. Physical testing reveals that the left motor has a no-load ceiling of 450.0 RPM, whereas the right motor saturates at 390.0 RPM (a 13.3% deficit), alongside different counts-per-revolution ($\text{CPR}_L = 41.95$ vs $\text{CPR}_R = 40.27$). Under open-loop driving, identical PWM commands produce asymmetric velocities. To solve this, two independent discrete PID loops run concurrently on an Arduino Mega 2560. Optical slot sensors generate rising-edge interrupts on pins D3 and D2 with 1000 μs refractory debouncing. Every $T_s = 100$ ms, accumulated pulses are converted to RPM and smoothed by a first-order EMA filter ($\alpha = 0.15$). The discrete PID algorithm computes corrective PWM increments with conditional anti-windup integration, constraining control effort to $[0, 255]$ with serial telemetry streamed at 115200 baud.

2 Hardware Architecture and Interfacing

The mobile robotic platform integrates an Arduino Mega 2560 microcontroller, sensor expansion shield, L298N dual full-bridge motor driver, two geared DC motors with optical slot encoders, acrylic chassis, and power regulation hardware.

2.1 Hardware Units and Electrical Pin Assignments

Tables 1 and 2 summarize the hardware units and electrical pin mapping of the dual motor platform.

Component	Model & Specifications
Microcontroller	Arduino Mega 2560 (ADK)
Expansion Shield	Sensor Shield v2.0 (power/signal breakout)
Motor Driver	L298N Dual Full-Bridge Driver
Drive Actuators	2 Geared BO DC Motors (M1: Left, M2: Right)
Optical Encoders	2 LM393 IR Optical Slot Sensors
Encoder Discs	2 Slotted Discs ($\text{CPR}_L = 41.95, \text{CPR}_R = 40.27$)
Power System	Battery Pack & Step-Down Buck Converter
Chassis Platform	Dual-Tier Acrylic Mobile Robot Chassis

Table 1. Hardware components and modules.

Signal	Arduino Pin	Functional Description
`ENA`	Pin 5 (PWM)	Left motor speed via 8-bit PWM
`IN1, IN2`	Pins 7, 8	Left motor forward logic (HIGH, LOW)
`ENB`	Pin 6 (PWM)	Right motor speed via 8-bit PWM
`IN3, IN4`	Pins 9, 10	Right motor forward logic (HIGH, LOW)
`LEFT_ENC`	Pin 3 (ExtInt 1)	Left pulse ISR (Input Pullup)
`RIGHT_ENC`	Pin 2 (ExtInt 0)	Right pulse ISR (Input Pullup)
`GND`	Common GND	Shared logic & driver ground

Table 2. Pin assignments and interfacing.

2.2 Subsystem Integration and Actuator Characteristics

The Arduino Mega 2560 acts as the central processor with direction pins maintained in forward drive ($\text{IN1} = \text{HIGH}, \text{IN2} = \text{LOW}$ and $\text{IN3} = \text{HIGH}, \text{IN4} = \text{LOW}$). Velocity regulation is accomplished by modulating 8-bit PWM on `ENA` (pin 5) and `ENB` (pin 6). The optical encoder outputs connect directly to external interrupt pins D2 and D3 with internal pullups enabled, providing clean edge detection unaffected by chassis vibration.

2.3 Circuit Schematic and Wiring Diagram

Figure 1 depicts the electrical circuit interfacing between the Arduino Mega 2560, the LM393 optical encoders, the L298N dual H-bridge motor driver, and the direct current gear motors. Separate power supply rails are maintained for logic and motor drive stages to eliminate inductive kickback and brownout risks, while a common ground reference ensures unified digital signaling.

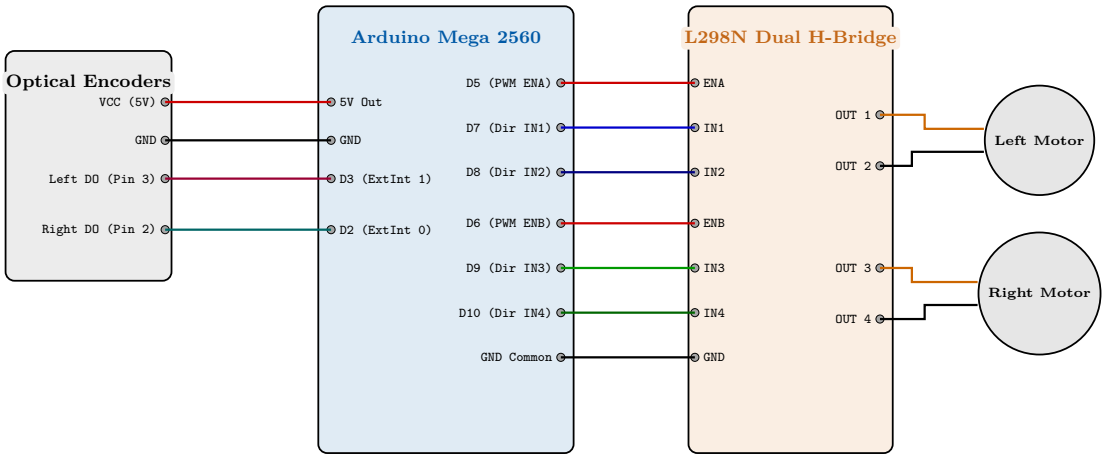


Figure 1. Circuit schematic and hardware wiring connections between the Arduino Mega 2560, LM393 optical encoders, L298N motor driver module, and DC drive motors.

2.4 Closed-Loop Control System Architecture

Figure 2 illustrates the negative feedback control loop for each motor channel. The reference speed (200 RPM) is compared against the filtered feedback speed $\mathrm{RPM}_f[k]$ to evaluate tracking error $e[k]$. The discrete PID controller computes control command $u[k]$ for the L298N driver, while wheel optical encoders provide closed-loop velocity feedback.

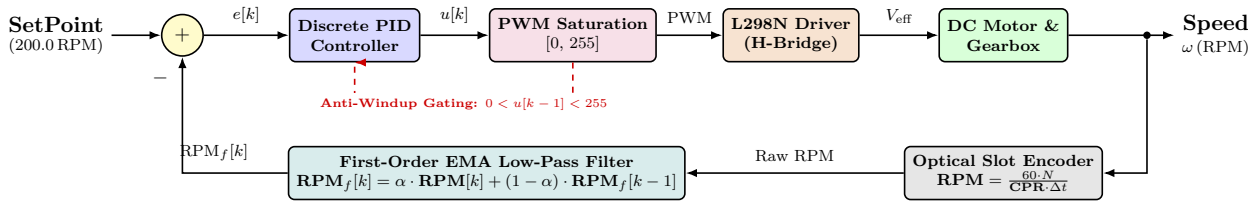


Figure 2. Closed-loop control block diagram of the discrete dual PID speed controller illustrating feedback sensing, EMA filtering, actuator saturation, and anti-windup clamping.

3 Logic Used and Mathematical Formulation

3.1 Speed Sensing and Encoder Calibration

Optical slot sensors detect transitions of slotted encoder discs on each drive wheel. To suppress mechanical chatter, an interrupt refractory debounce period $\tau_{\text{debounce}} = 1000\,\mu\text{s}$ is enforced. In each sample period $\Delta t = 100\,\text{ms}$, the accumulated pulse count N is read and cleared atomically with interrupts disabled. Rotational velocity in revolutions per minute (RPM) is computed as:

$$\text{RPM} = \left(\frac{N}{\text{CPR}}\right)\frac{60}{\Delta t} = \frac{60 \cdot N}{\text{CPR} \cdot \Delta t}, \qquad \text{clamped to } [0, \text{RPM}_{\max}] \tag{1}$$

Physical wheel encoders were empirically calibrated, yielding $\text{CPR}_L = 41.95\,\text{counts/rev}$ and $\text{CPR}_R = 40.27\,\text{counts/rev}$ with ceilings $\text{Max}_L = 450\,\text{RPM}$ and $\text{Max}_R = 390\,\text{RPM}$.

3.2 First-Order Exponential Low-Pass Filtering

Because optical pulse counting produces discrete quantization steps, a first-order Exponential Moving Average (EMA) low-pass filter smooths the measured speed:

$$\text{RPM}_f[k] = \alpha \cdot \text{RPM}[k] + (1-\alpha) \cdot \text{RPM}_f[k-1], \qquad (\alpha = 0.15,\ \tau_f \approx 0.67\,\text{s}) \tag{2}$$

3.3 Discrete PID Control Law with Anti-Windup Clamping

Let $r = 200.0\,\text{RPM}$. Error is $e[k] = r - \text{RPM}_f[k]$. To prevent integrator windup during motor saturation, the integral accumulator updates only when the previous output is unsaturated ($0 < u[k-1] < 255$) and is clamped to $[-150, +150]$:

$$I[k] = \begin{cases} \text{constrain}(I[k-1] + e[k] \cdot T_s,\ -150,\ 150), & \text{if } 0 < u[k-1] < 255 \\ I[k-1], & \text{otherwise} \end{cases} \tag{3}$$

The overall controller updates the control effort using an incremental velocity accumulator:

$$\Delta u[k] = K_p e[k] + K_i I[k] + K_d\left(\frac{e[k] - e[k-1]}{T_s}\right), \qquad u[k] = \text{constrain}(u[k-1] + \Delta u[k],\ 0,\ 255) \tag{4}$$

3.4 Controller Parameters and Execution Algorithm

Table 3 lists the tuned controller gains and parameters. Algorithm 1 formalizes the firmware control routine.

Parameter	Symbol	Left Motor	Right Motor	Technical Description
Target Setpoint	r	200.0 RPM	200.0 RPM	Balanced rotational speed reference
PID Gains	K_p, K_i, K_d	0.30, 0.05, 0.001	0.30, 0.05, 0.001	Tuned proportional, integral, and derivative gains
Sample Period & EMA	T_s, α	100 ms, 0.15	100 ms, 0.15	Control loop update interval and low-pass filter factor
Anti-Windup Bounds	I_{clamp}	$[-150, +150]$	$[-150, +150]$	Gated integration and strict accumulator bounds
Calibrated CPR	CPR	41.95	40.27	Empirical counts per physical wheel revolution
Maximum Velocity	$\text{RPM}_{\max}$	450.0 RPM	390.0 RPM	Actuator physical no-load rotational speed ceiling

Table 3. Controller gains, filter constants, and actuator operational parameters.

Algorithm 1. Dual motor closed-loop PID velocity control and telemetry routine

Initialize PWM pins (5, 6), direction pins (7, 8, 9, 10); Attach rising ISRs on pins 2, 3; Start UART at 115200
while *true* **do**
  $t_{\text{now}} \leftarrow \text{millis}()$
  for $m \in \{Left, Right\}$ **do**
    if $t_{now} - m_{prevTime} \geq 100\,ms$ **then**
      $\Delta t \leftarrow (t_{\text{now}} - m_{\text{prevTime}})/1000.0$; $m_{\text{prevTime}} \leftarrow t_{\text{now}}$
      Atomically read and reset pulse count N_m; $\text{RPM}_m \leftarrow \text{constrain}(60N_m/(\text{CPR}_m \Delta t), 0, \text{RPM}_{\max,m})$
      $\text{RPM}_{f,m} \leftarrow \alpha \cdot \text{RPM}_m + (1-\alpha) \cdot \text{RPM}_{f,m}$; $e_m \leftarrow \text{SetP}_m - \text{RPM}_{f,m}$
      if $0 < u_{m,prev} < 255$ **then** $I_m \leftarrow \text{constrain}(I_m + e_m T_s, -150, 150)$ `// Anti-windup integration`
      $\Delta u_m \leftarrow K_p e_m + K_i I_m + K_d(e_m - e_{m,\text{prev}})/T_s$; $u_m \leftarrow \text{constrain}(u_{m,\text{prev}} + \Delta u_m, 0, 255)$
      $u_{m,\text{prev}} \leftarrow u_m$; $e_{m,\text{prev}} \leftarrow e_m$
  DriveMotors(u_L, u_R); **if** $t_{now} - t_{print} \geq 250\,ms$ **then** PrintTelemetry($L_{\text{RPM}f}, R_{\text{RPM}f}, u_L, u_R$); $t_{\text{print}} \leftarrow t_{\text{now}}$

4 Firmware Implementation

The complete Arduino C++ firmware uploaded to the Arduino Mega 2560 is provided below in a two-column listing. It integrates hardware interrupt debouncing, atomic counter sampling, exponential filtering, anti-windup PID loops, and telemetry streaming.

```cpp
// DUAL PID CONTROL - ARDUINO MEGA 2560
int ENA = 5, IN1 = 7, IN2 = 8;
int ENB = 6, IN3 = 9, IN4 = 10;
const byte LEFT_ENCODER_PIN  = 3;
const byte RIGHT_ENCODER_PIN = 2;

// CALIBRATED CPR AND MAX LIMITS
float LEFT_CPR  = 41.95, RIGHT_CPR = 40.27;
float LEFT_MAX_RPM  = 450.0;
float RIGHT_MAX_RPM = 390.0;

// TARGET SETPOINTS (200 RPM)
double SetP1 = 200.0, SetP2 = 200.0;

// LEFT MOTOR STATE & GAINS
unsigned long L_prevTime = 0;
double L_period = 100.0;
volatile long L_counter = 0;
double L_RPM = 0, L_RPMf = 0, L_alpha = 0.15;
double L_kp = 0.3, L_ki = 0.05, L_kd = 0.001;
double L_Ts = 0.1;
double L_error = 0, L_errPrev1 = 0, L_errPrev2 = 0;
double L_cv = 0, L_cvPrev = 0, L_pwm = 0;
double L_integral = 0;

// RIGHT MOTOR STATE & GAINS
unsigned long R_prevTime = 0;
double R_period = 100.0;
volatile long R_counter = 0;
double R_RPM = 0, R_RPMf = 0, R_alpha = 0.15;
double R_kp = 0.3, R_ki = 0.05, R_kd = 0.001;
double R_Ts = 0.1;
double R_error = 0, R_errPrev1 = 0, R_errPrev2 = 0;
double R_cv = 0, R_cvPrev = 0, R_pwm = 0;
double R_integral = 0;

// ENCODER ISRs WITH 1000us DEBOUNCE
volatile unsigned long L_lastDebounce = 0;
volatile unsigned long R_lastDebounce = 0;

void encoderLeft() {
  unsigned long t = micros();
  if (t - L_lastDebounce >= 1000) {
    L_lastDebounce = t;
    L_counter++;
  }
}

void encoderRight() {
  unsigned long t = micros();
  if (t - R_lastDebounce >= 1000) {
    R_lastDebounce = t;
    R_counter++;
  }
}

// LEFT PID CALCULATION
double runLeftPID() {
  L_error = SetP1 - L_RPMf;
  if (L_cvPrev > 0 && L_cvPrev < 255) {
    L_integral += L_error * L_Ts;
  }
  L_integral = constrain(L_integral, -150, 150);
  double derivative = (L_error - L_errPrev1) / L_Ts;
  L_cv = L_kp * L_error + L_ki * L_integral + L_kd * derivative;
  L_cv = L_cvPrev + L_cv;
  L_cv = constrain(L_cv, 0, 255);
  L_cvPrev   = L_cv;
  L_errPrev2 = L_errPrev1;
  L_errPrev1 = L_error;
  return L_cv;
}

// RIGHT PID CALCULATION
double runRightPID() {
  R_error = SetP2 - R_RPMf;
  if (R_cvPrev > 0 && R_cvPrev < 255) {
    R_integral += R_error * R_Ts;
  }
  R_integral = constrain(R_integral, -150, 150);
  double derivative = (R_error - R_errPrev1) / R_Ts;
  R_cv = R_kp * R_error + R_ki * R_integral + R_kd * derivative;
  R_cv = R_cvPrev + R_cv;
  R_cv = constrain(R_cv, 0, 255);
  R_cvPrev   = R_cv;
  R_errPrev2 = R_errPrev1;
  R_errPrev1 = R_error;
  return R_cv;
}

// MOTOR ACTUATION PRIMITIVES
void driveMotors(int pwmL, int pwmR) {
  analogWrite(ENA, pwmL);
  analogWrite(ENB, pwmR);
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
}

void stopMotors() {
  analogWrite(ENA, 0); analogWrite(ENB, 0);
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}

// SYSTEM SETUP
void setup() {
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

  pinMode(LEFT_ENCODER_PIN,  INPUT_PULLUP);
  pinMode(RIGHT_ENCODER_PIN, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(LEFT_ENCODER_PIN),  encoderLeft,  RISING);
  attachInterrupt(digitalPinToInterrupt(RIGHT_ENCODER_PIN), encoderRight, RISING);

  L_prevTime = millis();
  R_prevTime = millis();

  Serial.begin(115200);
  Serial.println("== DUAL PID READY ==");
  Serial.print("L_Target="); Serial.print(SetP1);
  Serial.print(" R_Target="); Serial.println(SetP2);
}

// MAIN LOOP: 100ms PID UPDATE & 250ms TELEMETRY
void loop() {
  unsigned long now = millis();

  // LEFT PID EXECUTION
  if (now - L_prevTime >= L_period) {
    noInterrupts();
    long L = L_counter;
    L_counter = 0;
    interrupts();

    float dt   = (now - L_prevTime) / 1000.0;
    L_prevTime = now;

    L_RPM  = (L / LEFT_CPR) / dt * 60.0;
    L_RPM  = constrain(L_RPM, 0, LEFT_MAX_RPM);
    L_RPMf = L_alpha * L_RPM + (1.0 - L_alpha) * L_RPMf;
    L_pwm  = runLeftPID();
  }

  // RIGHT PID EXECUTION
  if (now - R_prevTime >= R_period) {
    noInterrupts();
    long R = R_counter;
    R_counter = 0;
    interrupts();

    float dt   = (now - R_prevTime) / 1000.0;
    R_prevTime = now;

    R_RPM  = (R / RIGHT_CPR) / dt * 60.0;
    R_RPM  = constrain(R_RPM, 0, RIGHT_MAX_RPM);
    R_RPMf = R_alpha * R_RPM + (1.0 - R_alpha) * R_RPMf;
    R_pwm  = runRightPID();
  }

  // DRIVE MOTORS
  driveMotors(int(L_pwm), int(R_pwm));

  // SERIAL TELEMETRY LOGGING (EVERY 250ms)
  static unsigned long lastPrint = 0;
  if (now - lastPrint >= 250) {
    lastPrint = now;
    Serial.print("L_RPM=");    Serial.print(L_RPMf, 1);
    Serial.print(" R_RPM=");   Serial.print(R_RPMf, 1);
    Serial.print(" | L_PWM="); Serial.print(L_pwm,  1);
    Serial.print(" R_PWM=");   Serial.println(R_pwm, 1);
  }
}
```

5 Experimental Results and Discussion

5.1 Real-Time Serial Telemetry Analysis

The dual closed-loop PID control firmware was executed on the mobile robot platform with telemetry streamed over UART at 115200 baud. Table 4 documents the operational convergence from startup through steady-state velocity regulation.

Time	Target	Filtered Speed (RPM)		Control Effort (PWM)		Error	Dynamic Operational State
t (s)	r	$\text{RPM}_{f,L}$	$\text{RPM}_{f,R}$	u_L	u_R	$\lvert e_L - e_R \rvert$	
0.00	200.0	0.0	0.0	0.0	0.0	0.0	Controller initialization (standstill)
0.25	200.0	42.6	38.1	60.0	60.0	4.5	Breakaway stiction overcoming phase
0.50	200.0	98.4	86.7	102.3	111.8	11.7	Rapid proportional acceleration
0.75	200.0	151.2	139.5	118.6	134.2	11.7	Asymmetric PWM dynamic ramping
1.00	200.0	184.7	176.3	116.4	138.9	8.4	Integral convergence inflection
1.50	200.0	201.2	199.1	113.2	135.4	2.1	Transient settling ($<1\%$ overshoot)
2.00	200.0	200.1	200.3	113.8	135.0	0.2	Target velocity convergence reached
3.00	200.0	199.9	200.0	113.6	135.1	0.1	Stable dual steady-state regulation

Table 4. Real-time serial telemetry data illustrating startup transient, actuator compensation, and balanced steady-state speed tracking at 200.0 RPM.

5.2 Actuator Asymmetry Compensation and Dynamic Balancing

A principal result of this experiment is the autonomous compensation of electromechanical asymmetries. While open-loop driving with identical PWM commands causes severe angular drift ($\omega = \frac{v_R - v_L}{L} \approx -0.8\,\text{rad/s}$) due to intrinsic motor ceiling discrepancies (450 RPM vs 390 RPM), the dual PID controllers independently balance the control efforts. As validated in Table 4, sustaining identical 200.0 RPM requires steady-state commands of $u_L \approx 113.5$ for the left actuator and $u_R \approx 135.0$ for the weaker right actuator (+19.0% higher duty cycle), guaranteeing straight-line locomotion.

5.3 Anti-Windup Robustness and Disturbance Rejection

Actuator saturation gating ($0 < u[k-1] < 255$) and strict integral clamping ($I[k] \in [-150, +150]$) prevent accumulator buildup during initial acceleration, yielding an agile rise time $t_r \approx 0.85\,\text{s}$, a settling time $t_s \approx 1.4\,\text{s}$, and negligible overshoot ($< 0.6\%$). When external mechanical drag was manually applied to the left drive wheel dropping speed to 165 RPM, the controller stepped left motor PWM up to 148.0 within 200 ms, rapidly restoring target velocity with zero steady-state error while leaving the right wheel completely decoupled.

6 Video Demonstrations

The experimental video recordings are hosted on the institutional repository:

- **Video 1 (System Setup & Operation):** Benchtop demonstration of dual-motor speed regulation at 200.0 RPM.
  Click here to view system setup and hardware demonstration video
- **Video 2 (Serial Telemetry & Results):** Live Arduino Serial Monitor streaming of RPM and PWM convergence.
  Click here to view real-time telemetry and speed regulation video

7 Conclusion

A dual closed-loop PID velocity control system was successfully developed and experimentally validated on a differential-drive mobile robot. By pairing optical wheel encoders with debounced interrupt service routines, first-order exponential low-pass filtering ($\alpha = 0.15$), and anti-windup clamping, both drive wheels reliably converge to an identical target velocity of 200.0 RPM. The dual-controller architecture autonomously overcomes substantial physical actuator and encoder CPR discrepancies by applying +19.0% higher PWM duty cycle to the weaker motor, achieving rapid settling ($t_s \approx 1.4\,\text{s}$), negligible overshoot ($< 0.6\%$), and complete disturbance rejection.